

```
$ R CMD check calculus
* using R version 4.1.2 (2021-11-01)
* using platform: x86_64-pc-linux-gnu (64-bit)
* using session charset: UTF-8
* checking compiled code ... OK
* checking installed files from 'inst/doc' ... OK
* checking files in 'vignettes' ... OK
* checking examples ... OK
* checking for unstated dependencies in 'tests' ... OK
* checking tests ...
  Running 'testthat.R'
OK
* checking for unstated dependencies in vignettes ... OK
* checking package vignettes in 'inst/doc' ... OK
* checking running R code from vignettes ...
  'derivatives.Rmd' using 'UTF-8'... OK
  'differential-operators.Rmd' using 'UTF-8'... OK
  'einstein.Rmd' using 'UTF-8'... OK
  'hermite.Rmd' using 'UTF-8'... OK
  'integrals.Rmd' using 'UTF-8'... OK
  'ode.Rmd' using 'UTF-8'... OK
  'taylor.Rmd' using 'UTF-8'... OK
NONE
* checking re-building of vignette outputs ... OK
* DONE
```

Status: OK

tinytest

The tinytest R package is meant to be a lightweight, no-dependency, complete package for unit testing. It is released under the GPLv3 license, and is meant to be easy to use. A list of available test functions provided by the package is given in the following table:

Function	Purpose
<i>expect_match</i>	A string regular expression match.
<i>expect_true</i>	Check that the argument is TRUE.
<i>expect_false</i>	Check that the argument is FALSE.
<i>expect_null</i>	Check that the expression is NULL.
<i>expect_equal</i>	The data and argument attributes must be equal.
<i>expect_inherits</i>	The object must inherit from the specific class.
<i>expect_error</i>	The expression must be an error.
<i>expect_warning</i>	The expression must be a warning.
<i>expect_message</i>	The expression must be a message.
<i>expect_silent</i>	There should not be an output.

The syntax for the test functions is similar to that of testthat. Consider the same square function once again:

```
> square <- function(x) return (x * x)
```

You can use the tinytest library's **expect_equal** assertion as follows:

```
> tinytest::expect_equal(square(2), 4)
----- PASSED : <-->
call| tinytest::expect_equal(square(2), 4)

> tinytest::expect_equal(square(2), 3)
----- FAILED[data]: <-->
call| tinytest::expect_equal(square(2), 3)
diff| Expected '3', got '4'
```

The objective here is to easily set up the test cases with simple R scripts. You can run the tests for the tinytest package from the R console itself, and produce a summary output as shown below:

```
> library(tinytest)

> out <- run_test_dir(system.file("tinytest",
package="tinytest"), verbose=0)

> summary(out)
```

File	Results	fails	passes
test_call_wd.R	1	0	1
test_env_A.R	2	0	2
test_env_B.R	6	0	6
test_extensibility.R	1	0	1
test_file.R	23	0	23
test_gh_issue_17.R	2	0	2
test_gh_issue_32.R	3	0	3
test_gh_issue_51.R	1	0	1
test_gh_issue_58.R	2	0	2
test_gh_issue_86.R	4	0	4
test_init.R	1	0	1
test_RUnit_style.R	5	0	5
test_tiny.R	74	0	74
test_utils.R	4	0	4
Total	129	0	129

You are encouraged to read the reference manuals for the above unit testing frameworks, and use them in your R packages. 

References

- [1] RUnit - A Unit Test Framework in R, <https://cran.r-project.org/web/packages/RUnit/vignettes/RUnit.pdf>
- [2] Testthat, <https://testthat.r-lib.org/reference/index.html>
- [3] Tinytest, <https://github.com/markvanderloo/tinytest>
- [4] Using Tinytest, https://cran.r-project.org/web/packages/tinytest/vignettes/using_tinytest.pdf

 By: Shakthi Kannan

The author is a free software enthusiast.